Execution-speed experiments

Mathilda — execution-speed experiments

Contents

Execution-speed experiments, 2026-07-26 → 2026-07-31

Twenty experiments, run over six days, each aimed at one hypothesis about why Mathilda was slower than it should be. One file per experiment. Every file reports the same three columns — Mathilda, Mathematica 14.0, and Python (NumPy/SciPy where the kernel is vectorisable, CPython otherwise) — because the two comparisons answer different questions:

- Mathematica says whether Mathilda is behind a competitor: a mature CAS with the same evaluation semantics, the same automatic compilation, and the same automatic packed arrays. A gap here is a gap in engineering.
- NumPy says whether Mathilda is behind the machine. On this host NumPy links the same Apple Accelerate BLAS that Mathilda does, so on the dense rows all three call byte-identical kernels and any spread is pure overhead. On the array rows NumPy is a thin memory-bandwidth reference.

A row can read acceptably against one column and badly against the other, and that difference is usually the finding. The sieve is 1.18× ahead of Mathematica and 25× behind NumPy; before the fourth sweep the return-series kernel was 13.5× behind Mathematica and 66.5× behind NumPy. Only the second number separates ”slower than a competitor” from ”slower than a memory copy”.

Where NumPy has no equivalent (arbitrary precision, symbolic rule dispatch, `PrimePi`) the column reads –. Where the honest Python answer is a CPython loop or `mpmath` rather than a library call, the row says so — numba is not installed on this host, and those rows must not be read as library results.

Method

Common to every experiment; stated once here rather than nineteen times.

- Machine: Intel Core i9-9880H (8C/16T, AVX2), 16 GB, macOS 15.7.4.
- Versions: Mathilda at −O3 with GMP 6.3, MPFR 4.2.2, FLINT 3.6, FFTW 3.3.11, Apple Accelerate. Mathematica 14.0.0. NumPy 2.4.4 / SciPy 1.17.1 on CPython 3.11, also linked against Accelerate.
- Wall clock: `AbsoluteTiming` in both CAS, `time.perf_counter` in Python. Never `Timing[]`, which sums CPU time over threads and so overstates a threaded path by roughly the core count.

- Minimum of several repetitions after one untimed warm-up, with the maximum recorded as a caching tripwire.
- The same source text runs in Mathilda and Mathematica; the Python column is a separate implementation of the same algorithm in the same order.
- Values are checked, not just timed. Each benchmark has a cheap scalar answer recorded from all three systems and compared. A timing row is meaningless until they agree — the check pins the algorithm, so the columns cannot silently be timing three different computations.
- One experiment per process (from the fifth sweep on). The harness shares a single kernel session across everything it is given, and with eight domains' worth of setup data co-resident the fifth sweep's first attempt was killed by the OOM killer at 21.7 minutes. Each group now runs in its own process.
- Before/after inside Mathilda is a rebuild, not a memory. The "before" binary is this tree with the sweep's patches reverted, built and run on the same host on the same day; the kill switches (`MATHILDA_NO_PACK`, `MATHILDA_NO_NUMLOOP`, `MATHILDA_NO_AUTOCOMPILE`, `$AutoArrayPacking`, `$AutoCompilation`) are used for the within-binary comparisons.
- Rows whose working set is hundreds of megabytes carry ±20% run-to-run spread in the harness, from allocator and thread state left by the rows before them. Where a before/after difference is inside that band it is checked by a standalone A/B rather than reported — see `13-molecular-dynamics/`.

Reproduce with `comparisons/hpc_bench.py`.

How each experiment is laid out

One folder per experiment, and the same three files in each:

```
NN-slug/
  README.md      the write-up: what was measured, what it found, and -- for
                 every row where Mathilda is not the fastest of the three --
                 why, and what it would take to lead
  <slug>.m       the kernels, runnable UNMODIFIED in Mathilda and in
                 Mathematica: one source text, so the two CAS columns cannot
                 silently be timing different programs
  <slug>.py      the same algorithm in NumPy, in the same order
  <slug>.pdf     the write-up, rendered
```

Run one, or all three side by side:

```
docs/experiments/run.sh 12-graph-and-sparse              # all three systems
docs/experiments/run.sh 12-graph-and-sparse mathilda     # just one
docs/experiments/build_pdfs.sh                           # re-render the PDFs
```

`run.sh` only lines the three columns up side by side; every system also takes the file directly, and the `.m` is the same text in both CAS columns:

```
./Mathilda -file docs/experiments/12-graph-and-sparse/graph_and_sparse.m
wolframscript -file docs/experiments/12-graph-and-sparse/graph_and_sparse.m
python3 docs/experiments/12-graph-and-sparse/graph_and_sparse.py
```

Where a `.py` file is not a library result it says so in its own docstring: `numba` is not installed on this host, so the scalar-loop kernels of experiments 1, 3 and 5 are plain CPython and must be read as "an interpreted scalar loop", not as "what Python can do".

The experiments

#	Experiment	Dates	Headline
1	`01-compile-engine/`	07-26 → 07-28	A typed bytecode VM for numeric code: ~234× over the inter
2	`02-compile-array-fusion/`	07-27	Machine arrays and fused elementwise loops: 3.2–6.6× at 10
3	`03-compile-optimising-codegen/`	07-27	CSE, folding, DCE, LICM, threaded dispatch: 1.5× on top

#	Experiment	Dates	Headline
4	04-auto-compilation/	07-26 → 07-29	The compiler runs without Compile[]: 6×–353×
5	05-machine-integers/	07-29 → 07-30	int64 as a peer of float64, without losing exactness
6	06-packed-arrays/	07-30	Dense lists become machine buffers invisibly: up to 56000×
7	07-blas-lapack-routing/	07-30	Reaching the vendor kernels, and a rejected scratch pool
8	08-random-number-generation/	07-30	A machine-precision generator: 53×–70×
9	09-hpc-sweep-primitives/	07-30	43 classical kernels against Mathematica; what it exposed
10	10-hpc-sweep-applications/	07-31	Eight real-application pipelines, three systems
11	11-hpc-sweep-numpy-gap/	07-31	Closing on NumPy: structural ops, scans, convolution
12	12-graph-and-sparse/	07-31	PageRank 14.1 s → 486 ms: nobody had packed the index
13	13-molecular-dynamics/	07-31	Lennard-Jones 6.9× ahead of Mathematica, and a return that
14	14-sequence-alignment/	07-31	Needleman-Wunsch 60×: the integer half of the buffer was m
15	15-option-pricing/	07-31	92× and 135×: the positive part had no working spelling
16	16-spectral-pde/	07-31	FFT in a time loop: 18.4× ahead of Mathematica, no fix need
17	17-neural-network-training/	07-31	123× behind on one missing function; inference now ahead o
18	18-state-estimation/	07-31	The packing threshold is a floor under numeric linear algebra
19	19-ray-tracing/	07-31	7.4× ahead of Mathematica; a 65-element table cost 17×
20	20-numeric-coverage-sweep/	07-31	Every builtin, not every workload: MinMax 636×, the predicat

Experiments 12–19 are one sweep, run together: eight application domains chosen so that each one goes somewhere the first four sweeps did not. The selection rule was mechanical — for each candidate, name the subsystem it would exercise that no existing row does, and drop anything that could not answer.

Experiment 20 is the first that is not workload-driven at all. Every one of the nineteen before it chose kernels and saw which builtins they reached, which means the coverage is real but accidental: nothing here could answer "which builtins have never been measured?" Experiment 20 inverts that — it enumerates all 676 registered builtins, joins them against the registries that decide dispatch, and times the numeric ones against NumPy/SciPy. It found the same defect the nineteen kept finding (MinMax, 636×) by enumeration rather than by accident, and left a register of 42 measured gaps and 96 absent functions.

The roadmap, across all nineteen

Every experiment ends with a section naming, for each row where Mathilda is not the fastest of the three, why and what it would take. Collected here in value order, because the same handful of items keeps appearing:

#	item
1	A heterogeneous packed tuple — a container holding n buffers of independent shape and dtype without materialising th
2	Strided views — a stride vector on NDArray that consumers honour, giving a free Transpose and a free sliding window
3	Route small machine matrices to LAPACK — dispatch on "is this a matrix of machine numbers", not "is this already a bu
4	Interpreter-level fusion (plan 9.2) — Compile[] fuses, ordinary array code does not
5	Reach the remaining LAPACK drivers — QRDecomposition, Eigenvalues, SVD
6	A **start/step/n** selector — no position array for spans, strided writes or gathers
7	Cut the per-operation constant (plan 5.2) — ~8 µs per array operation, which dominates any loop with a short body
8	A vector math library for the transcendentals — Accelerate's vv∗, already linked
9	Thread the block moves and the gather — Reverse, RotateLeft, Part
10	Vectorise the elementwise binary kernels — MapThread[Min] runs at 7.7 GB/s where NumPy reaches 48

Items 1, 2 and 3 are design changes with measured payoffs. Items 4 and 7 are the two that appear most often. Items 5, 6, 8, 9 and 10 are self-contained kernel work with no open question attached.

plans/HPC_IMPROVEMENT_PLAN.md sequences these by value and risk; Phase 10 carries the fifth sweep's share.

The one finding that recurs

Six of the seven fixes in experiment 10, four of the six in experiment 11, and five of the nine in the fifth sweep are the same defect wearing different clothes:

> An operation had a working buffer path and a working list path, and quietly took the second whenever the two representations met.

The packing threshold judges each value in isolation, but a binary operation's cost is set by its largest operand. The rule that resolves it, now applied at ten sites:

> Pack the small operand up; never materialise the large one down.

The fifth sweep found three new faces of it, and they are worth naming separately because none is a "small operand" in the obvious sense:

- The index of a gather. `x[[idx]]` is a large operand meeting a large one, but the index arrived packed and the selector only accepted a plain `List`, so both were materialised — PageRank at 40× behind Mathematica (experiment 12).
- A lookup table. 65 entries is small by design, not by accident, and no threshold will ever pack it — so the gather has to lift it (experiment 19).
- A value built before the loop. One unpacked vector created in a setup line cost 25000 iterations their fast path, because every iteration met it (experiment 15).

And a fourth that is the same shape and has no fix yet: a function returning `{a, b, mask}` destroys all three arrays, because one of them is an integer buffer and a mixed-dtype list of packed rows cannot be absorbed — 96× at the **`return`**, 120× on the caller's next operation (experiment 13).

The second finding: fix the primitive, then re-profile the composition

Experiment 14 made `MapThread` 62× faster and `FoldList` 222× faster on integer data, and the benchmark built out of both moved 5% — because `Abs` and `Most` upstream of them were still materialising, and were handing the fast primitives unpacked operands. Two further rounds of profiling took that row to 60×.

The fourth sweep hit this three times (a correct `Outer` fast path that was unreachable; a correct broadcast that never fired) and the third twice. The only reliable protocol is to re-measure the composition after fixing the primitive, and to treat a primitive win that does not show up downstream as evidence rather than as noise.

The third: tests below the threshold test nothing

Third and fourth appearance of the same lesson. Every differential case added by the fifth sweep evaluates the same source twice — once with automatic packing on, once with it off — so what is compared is the new buffer path against the interpreter's List path on identical input, rather than against a written-down expectation that can be wrong in the same direction as the code. Every source is above `PACK_MIN_ELEMENTS`, and every form each path must decline is included.

See also

- `docs/design/performance.md` — the combined three-system table across all benchmarks.
- `docs/design/compile.md` — the compiler's design.
- `docs/design/packed_arrays.md` — the packing model.
- `plans/HPC_IMPROVEMENT_PLAN.md` — what each sweep left open.